

- Uma classe derivada pode efetuar mudanças de estado nos membros `private` da classe-base, mas somente por meio de funções-membro não `private` fornecidas na classe-base e herdadas para a classe derivada.
- Uma função-membro da classe-base pode ser redefinida em uma classe derivada.
- Os relacionamentos de herança simples formam estruturas hierárquicas do tipo árvore.
- É possível tratar objetos da classe-base e objetos da classe derivada de modo semelhante; a semelhança compartilhada entre os tipos de objeto é expressa nos dados-membro e nas funções-membro da classe-base.

### Seção 20.3 Membros `protected`

- Os membros `public` da classe-base são acessíveis de qualquer lugar em que o programa tenha um handle para um objeto dessa classe-base ou para um objeto de uma das classes derivadas dessa classe-base — ou, ao usar o operador binário de resolução de escopo sempre que o nome da classe estiver no escopo.
- Os membros `private` de uma classe-base são acessíveis apenas dentro da classe-base ou de suas `friends`.
- Os membros `protected` de uma classe-base podem ser acessados por membros e `friends` dessa classe-base e por membros e `friends` de quaisquer classes derivadas dessa classe-base.
- Quando uma função-membro de classe derivada redefine uma função-membro de classe-base, a função-membro da classe-base pode ser acessada a partir da classe derivada qualificando o nome da função-membro da classe-base com o nome da classe-base e o operador binário de resolução de escopo (`::`).

### Seção 20.5 Construtores e destrutores em classes derivadas

- Quando um objeto de uma classe derivada é instanciado, o construtor da classe-base é chamado imediatamente para inicializar os dados-membro da classe-base no objeto da classe derivada, depois o construtor da classe derivada inicializa os dados-membro adicionais da classe derivada.
- Quando um objeto da classe derivada é destruído, os destrutores são chamados na ordem contrária à ordem dos construtores — primeiro, o destrutor da classe derivada é chamado, e, depois, o destrutor da classe-base é chamado.

### Seção 20.6 Heranças `public`, `protected` e `private`

- Ao declararmos os dados-membro `private` enquanto oferecemos funções-membro não `private` para manipular e realizar a verificação de validade nesses dados, impomos a boa engenharia de software.
- Ao derivar uma classe, a classe básica pode ser declarada como `public`, `protected` ou `private`.
- Ao derivar uma classe de uma classe-base `public`, os membros `public` da classe-base tornam-se membros `public` da classe derivada, e os membros `protected` da classe-base tornam-se membros `protected` da classe derivada.
- Ao derivar uma classe de uma classe-base `protected`, os membros `public` e `protected` da classe-base tornam-se membros `protected` da classe derivada.
- Ao derivar uma classe de uma classe-base `private`, membros `public` e `protected` da classe-base tornam-se membros `private` da classe derivada.

## Terminologia

classe derivada 615

classe-base 615

classe-base direta 615

classe-base indireta 615

herança 615

herança múltipla 615

herança `private` 618

herança `protected` 618

herança `public` 618

herança simples 615

herdar 615

hierarquia de classes 615

palavra-chave, `protected` 618

relacionamento *é-um* 615

relacionamento *tem-um* 615

sintaxe de inicializador de classe-base 630

software frágil 637

software quebradiço 637

subclasse 615

superclasse 615

## Exercícios de autorrevisão

**20.1** Preencha os espaços em cada uma das sentenças:

- a)** \_\_\_\_\_ é uma forma de reutilização de software em que novas classes absorvem dados e comportamentos das classes existentes e aprimoram essas classes com novas capacidades.

- b)** Os membros \_\_\_\_\_ de uma classe-base podem ser acessados na definição da classe-base, nas definições da classe derivada e em `friends` das classes derivadas dessa classe-base.

- c) Em um relacionamento \_\_\_\_\_, um objeto de uma classe derivada também pode ser tratado como um objeto de sua classe-base.
  - d) Em uma relação \_\_\_\_\_, um objeto de classe tem um ou mais objetos de outras classes como membros.
  - e) Na herança simples, uma classe existe em um relacionamento \_\_\_\_\_ com suas classes derivadas.
  - f) Os membros \_\_\_\_\_ de uma classe-base são acessíveis dentro dessa classe-base e em qualquer lugar em que o programa tenha um handle para um objeto dessa classe ou uma de suas classes derivadas.
  - g) Os membros de acesso `protected` de uma classe-base possuem um nível de proteção entre aqueles de acesso `public` e \_\_\_\_\_.
  - h) C++ oferece \_\_\_\_\_, que permite que uma classe derivada herde de muitas classes-base, mesmo que as classes-base não estejam relacionadas.
  - i) Quando um objeto de uma classe derivada é instanciado, o(a) \_\_\_\_\_ da classe-base é chamado(a) implícita ou explicitamente para realizar qualquer inicialização necessária dos dados-membro da classe-base no objeto da classe derivada.
  - j) Ao derivar uma classe de uma classe-base com herança `public`, os membros `public` da classe-base tornam-se membros \_\_\_\_\_ da classe derivada, e os membros `protected` da classe-base tornam-se membros \_\_\_\_\_ da classe derivada.
  - k) Ao derivar uma classe de uma classe-base com herança `protected`, os membros `public` da classe-base tornam-se membros \_\_\_\_\_ da classe derivada, e os membros `protected` da classe-base tornam-se membros \_\_\_\_\_ da classe derivada.
- 20.2** Indique se cada uma das sentenças a seguir é *verdadeira* ou *falsa*. Em caso de alternativas falsas, justifique sua resposta.
- a) Os construtores da classe-base não são herdados pelas classes derivadas.
  - b) Um relacionamento *tem-um* é implementado por meio de herança.
  - c) Uma classe Carro tem um relacionamento *é-um* com as classes Direção e Freios.
  - d) A herança encoraja a reutilização de software de alta qualidade comprovada.
  - e) Quando um objeto da classe derivada é destruído, os destrutores são chamados na ordem contrária à dos construtores.

## ■ Respostas dos exercícios de autorrevisão

- 20.1** a) Herança. b) `protected`. c) *é-um* ou herança. d) *tem-um*, composição ou agregação. e) hierárquico. f) `public`. g) `private`. h) herança múltipla. i) construtor. j) `public`, `protected`. k) `protected`, `protected`.
- 20.2** a) Verdadeiro. b) Falso. Um relacionamento *tem-um* é implementado por meio de composição. Um relacionamento *é-um* é implementado por meio de herança. c) Falso. Este é um exemplo de um relacionamento *tem-um*. A classe Carro tem um relacionamento *é-um* com a classe Veículo. d) Verdadeiro. e) Verdadeiro.

## ■ Exercícios

- 20.3** *Composição como alternativa à herança.* Muitos programas escritos com herança poderiam ser escritos com composição, e vice-versa. Reescreva a classe `FuncionarioBaseMaisComissao` da hierarquia `FuncionarioComissao`–`FuncionarioBaseMaisComissao` para usar a composição em vez da herança. Depois que você fizer isso, avalie os méritos relativos dos dois métodos para projetar as classes `FuncionarioComissao` e `FuncionarioBaseMaisComissao`, bem como para programas orientados a objeto em geral. Qual método é mais natural? Por quê?
- 20.4** *Vantagem da herança.* Discuta as maneiras como a herança promove a reutilização de software, economiza tempo no desenvolvimento do programa e ajuda a impedir a ocorrência de erros.
- 20.5** *Classes-base protegidas versus classes-base privadas.* Alguns programadores preferem não usar o acesso `protected` porque acreditam que isso quebra o encapsulamento da classe-base. Discuta os méritos relativos do uso do acesso `protected` e do uso do acesso `private` nas classes-base.

**20.6 Hierarquia de herança de estudante.** Desenhe uma hierarquia de herança para os estudantes em uma universidade, semelhante à hierarquia mostrada na Figura 20.2. Use *Estudante* como classe-base da hierarquia, e depois inclua as classes *EstudanteCursando* e *EstudanteFormado* que derivam de *Estudante*. Continue a expandir a hierarquia com a maior profundidade (ou seja, com o máximo de níveis) possível. Por exemplo, *Calouro*, *Estudante do segundo ano*, *Junior* e *Sênior* poderiam derivar de *EstudanteCursando*, e *EstudanteDoutorado* e *EstudanteMestrado* poderiam derivar de *EstudanteFormado*. Depois de desenhar a hierarquia, discuta as relações existentes entre as classes. [Nota: você não precisa escrever código algum nesse exercício.]

**20.7 Hierarquia de forma mais rica.** O mundo das formas é muito mais rico que as formas incluídas na hierarquia de herança da Figura 20.3. Escreva todas as formas as quais você puder lembrar — tanto bidimensionais quanto tridimensionais — e componha em uma hierarquia *Forma* mais completa, com o máximo possível de níveis. Sua hierarquia deverá ter a classe-base *Forma*, da qual as classes *FormaBidimensional* e *FormaTridimensional* sejam derivadas. [Nota: você não precisa escrever código algum nesse exercício.] Usaremos essa hierarquia nos exercícios do Capítulo 21 para processar um conjunto de formas distintas como objetos da classe-base *Forma*. (Essa técnica, chamada de polimorfismo, será o assunto do Capítulo 21.)

**20.8 Hierarquia de herança de quadrilátero.** Desenhe uma hierarquia de herança para as classes *Quadrilátero*, *Trapezoide*, *Paralelogramo*, *Retângulo* e *Quadrado*. Use *Quadrilátero* como classe-base da hierarquia, que deverá ser tão profunda quanto possível.

**20.9 Hierarquia de herança de pacote.** Os serviços de entrega de pacotes, como *FedEx*®, *DHL*® e *UPS*®, oferecem uma série de opções diferentes de entrega, cada uma com custos específicos. Crie uma hierarquia de herança que represente os diversos tipos de pacotes. Use *Pacote* como classe-base da hierarquia, e depois inclua as classes *PacoteDoisDias* e *PacoteNoturno* que derivem de *Pacote*. A classe-base *Pacote* deverá incluir dados-membro que representem nome, endereço, cidade, estado e CEP do remetente e do destinatário do pacote, além de dados-membro que armazenem o peso (em gramas) e o custo por grama para enviar o pacote. O construtor do *Pacote* deverá inicializar esses dados-membro. Cuide para que o peso e o custo por grama conttenham valores positivos. O *Pacote* deverá

fornecer uma função-membro `public calculaCusto` que retorne um `double` indicando o custo associado à remessa do pacote. A função `calculaCusto` de *Pacote* deverá determinar o custo multiplicando o peso pelo custo por grama. A classe derivada *PacoteDoisDias* deverá herdar a funcionalidade da classe-base *Pacote*, mas também deverá incluir um dado-membro que represente um valor fixo cobrado pela empresa para prestar um serviço de entrega em dois dias. O construtor de *PacoteDoisDias* deverá receber um valor para inicializar esse dado-membro. *PacoteDoisDias* deverá redefinir a função-membro `calculaCusto` de modo que calcule o custo da entrega somando o valor fixo ao custo baseado no peso, calculado pela função `calculaCusto` do *Pacote*. A classe *PacoteNoturno* deverá herdar diretamente da classe *Pacote* e conter um dado-membro adicional que represente uma taxa adicional por grama cobrada para o serviço de entrega noturna. *PacoteNoturno* deverá redefinir a função-membro `calculaCusto` de modo que some a taxa adicional por grama ao custo-padrão por grama antes de calcular o custo da entrega. Escreva um programa de teste que crie objetos de cada tipo de *Pacote* e teste a função-membro `calculaCusto`.

**20.10 Hierarquia de herança de Conta.** Crie uma hierarquia de herança que pudesse ser usada por um banco para representar as contas bancárias dos clientes. Todos os clientes desse banco poderiam depositar (ou seja, creditar) dinheiro em suas contas e sacar (ou seja, debitar) dinheiro de suas contas. Também existiriam tipos mais específicos de contas. Contas-poupança, por exemplo, receberiam juros sobre o dinheiro mantido. Contas-correntes, por outro lado, cobrariam uma taxa por transação (ou seja, crédito ou débito).

Crie uma hierarquia de herança que contenha a classe-base *Conta* e as classes derivadas *ContaPoupança* e *ContaCorrente*, que herdem da classe *Conta*. A classe-base *Conta* deverá incluir um dado-membro do tipo `double` para representar o saldo da conta. A classe deverá fornecer um construtor que receba um saldo inicial e o utilize para inicializar o dado-membro. O construtor deverá validar o saldo inicial para garantir que ele seja maior ou igual a 0.0. Se não for, o saldo deverá ser definido como 0.0, e o construtor deverá exibir uma mensagem de erro que indique que o saldo inicial é inválido. A classe deverá fornecer três funções-membro. A função-membro `creditar` deverá somar um valor ao saldo atual. A função-membro `debitar` deverá retirar dinheiro da *Conta* e garantir que o valor do débito não

ultrapasse o saldo da Conta. Se isso acontecer, o saldo deverá ficar inalterado e a função deverá imprimir a mensagem “Valor do débito superior ao saldo da conta”. A função-membro `retSaldo` deverá retornar o saldo atual.

A classe derivada `ContaPoupança` deverá herdar a funcionalidade de uma `Conta`, mas também deverá incluir um dado-membro do tipo `double` que indique a taxa de juros (porcentagem) atribuída à `Conta`. O construtor da `ContaPoupança` deverá receber o saldo inicial, bem como um valor inicial para a taxa de juros da `ContaPoupança`. A `ContaPoupança` deverá fornecer uma função-membro `public calculaJuros`, que retorne um `double` indicando o valor dos juros recebidos por uma conta. A função-membro `calculaJuros` deverá determinar esse valor multiplicando a taxa de juros pelo saldo da conta. [Nota: `ContaPoupança` deverá herdar as funções-membro `creditar` e `debitar` como se encontra, sem redefini-las.]

A classe derivada `ContaCorrente` deverá herdar da classe-base `Conta` e incluir um dado-membro adicional do tipo `double`, que represente uma taxa cobrada por

transação. O construtor de `ContaCorrente` deverá receber o saldo inicial, bem como um parâmetro indicando um valor de taxa. A classe `ContaCorrente` deverá redefinir as funções-membro `creditar` e `debitar`, de modo que elas subtraíam a taxa do saldo da conta sempre que alguma transação for realizada com sucesso. As versões de `ContaCorrente` dessas funções deverão chamar a versão `Conta` da classe-base para realizar as atualizações no saldo de uma conta. A função `debit` de `ContaCorrente` deverá cobrar uma taxa somente se o dinheiro for realmente retirado (ou seja, se o valor de débito não ultrapassar o saldo da conta). [Dica: defina a função `debitar` de `Conta` de modo que ela retorne um `booleano` que mostre se o dinheiro foi retirado. A seguir, use o valor de retorno para determinar se uma taxa deverá ser cobrada.]

Depois de definir as classes nessa hierarquia, escreva um programa que crie objetos de cada classe e teste suas funções-membro. Some juros ao objeto `ContaPoupança`, chamando sua função `calculaJuros` em primeiro lugar, e, depois, passando o valor de juros retornado à função `creditar` do objeto.